UNIVERSIDAD NACIONAL DE COLOMBIA

Facultad de Ingeniería y Arquitectura

Modelado y Control de Robots Móviles

Informe de Laboratorio – Capítulo 6 (Corke)

Localización y Mapeo

Curso: Robótica
Código: MAN-4101136-1-2026-03

Integrantes:

Lina Robsana Jamioy Tisoy
Tomás Rocha Castaño

Profesor:
Jaime Enrique Arango Castro

12 de noviembre de 2026

Índice

1. Objetivos **2**

2. Marco teórico **2**

3. Entorno de trabajo **2**

4. Parte 1: Odometría (Dead Reckoning) **4**
4.1. ¿Qué hicimos? . . . 4
4.2. Comando y código . . . 4
4.3. Resultado . . . 5
4.4. Preguntas de análisis . . . 5

5. Parte 2: Estimación con EKF **7**
5.1. ¿Qué hicimos? . . . 7
5.2. Comando y código . . . 7
5.3. Resultado . . . 8
5.4. Preguntas de análisis . . . 9

6. Parte 3: EKF con landmarks **10**
6.1. ¿Qué hicimos? . . . 10
6.2. Comando y código . . . 10
6.3. Resultado . . . 11
6.4. Preguntas de análisis . . . 12

7. Parte 4: SLAM con EKF **14**
7.1. ¿Qué hicimos? . . . 14
7.2. Comando y código . . . 14
7.3. Resultado . . . 15
7.4. Preguntas de análisis . . . 16

8. Parte 5: SLAM con Cartographer (ROS 2 Jazzy, TurtleBot3, Gazebo y RViz2) **17**
8.1. ¿Qué hicimos? . . . 17
8.2. Carpetas del laboratorio y capturas . . . 17
8.3. Pasos 8.1 a 8.6: verificar el entorno . . . 17
8.4. Pasos 8.7 a 8.9: workspace y paquete . . . 20
8.5. Paso 8.10: el controlador (teleoperación) . . . 20
8.6. Pasos 8.11 a 8.14: archivos del paquete . . . 21
8.7. Pasos 8.15 a 8.17: compilar y configurar el robot . . . 22
8.8. Pasos 8.18 a 8.23: ejecución . . . 23
8.9. Mapeo, errores y cambios . . . 28
8.10. Pasos 8.24 y 8.25: resultado y guardado del mapa . . . 29
8.11. Evidencia en video y código . . . 30
8.12. Preguntas de análisis . . . 31
8.13. Paso 8.26: concepto central . . . 31

9. Discusión comparativa **32**

10. Conclusiones **32**

1. Objetivos

Al terminar este laboratorio, el estudiante será capaz de:

1. Entender cómo un robot estima su posición usando solo sus ruedas (odometría) y por qué esa estimación se equivoca con el tiempo.
2. Usar un filtro de Kalman extendido (EKF) para estimar la posición del robot.
3. Corregir la estimación usando marcas conocidas del entorno (landmarks).
4. Entender SLAM, que consiste en construir un mapa y ubicarse en él al mismo tiempo.

2. Marco teórico

Estos son los conceptos principales, explicados de forma sencilla:

- **Localización:** saber dónde está el robot dentro de un mapa o de un sistema de coordenadas.
- **Odometría:** calcular la posición contando cuánto han girado las ruedas. Es como caminar con los ojos cerrados contando los pasos: cada paso tiene un pequeño error, y los errores se acumulan.
- **Filtro de Kalman extendido (EKF):** combina dos cosas: lo que el robot *predice* que hizo (por la odometría) y lo que *mide* con sus sensores. Además, calcula qué tan seguro está de su estimación. Esa medida de seguridad es la **incertidumbre**: mientras más alta, menos seguro está el robot de dónde se encuentra.
- **Landmarks:** marcas conocidas del entorno. El sensor mide la distancia y el ángulo hacia cada marca, y con esa información el filtro corrige su estimación.
- **SLAM:** *Simultaneous Localization and Mapping*. El robot no conoce el mapa al inicio; lo construye mientras se mueve y, al mismo tiempo, se ubica en él.

3. Entorno de trabajo

Trabajamos con los siguientes sistemas y versiones:

- **Sistema operativo:** Ubuntu 24.04.5 LTS
- **Python:** 3.12.3
- **Librerías:** `numpy` 1.26.4, `matplotlib` 3.6.3 y `roboticstoolbox-python`
- **ROS 2 (parte 5):** Jazzy Jalisco, con Gazebo Harmonic, TurtleBot3 (modelo *burger*), Cartographer y Nav2

Para no mezclar librerías, creamos un entorno virtual de Python (venv) dentro de la carpeta del laboratorio. Los comandos usados fueron:

```
lsb_release -a
echo $ROS_DISTRO
python3 --version
cd ~/lab_cap6
python3 -m venv venv
source venv/bin/activate
pip install roboticstoolbox-python matplotlib numpy
```

```
python -c "from roboticstoolbox import Bicycle, EKF, RandomPath, LandmarkMap,
    RangeBearingSensor; print('Importacion OK')"
```

Listing 1: Comandos de verificación del entorno

La salida confirma la versión de Ubuntu, la distribución de ROS 2 (`jazzy`), Python 3.12.3 y que todas las clases necesarias de `roboticstoolbox` se importan correctamente.

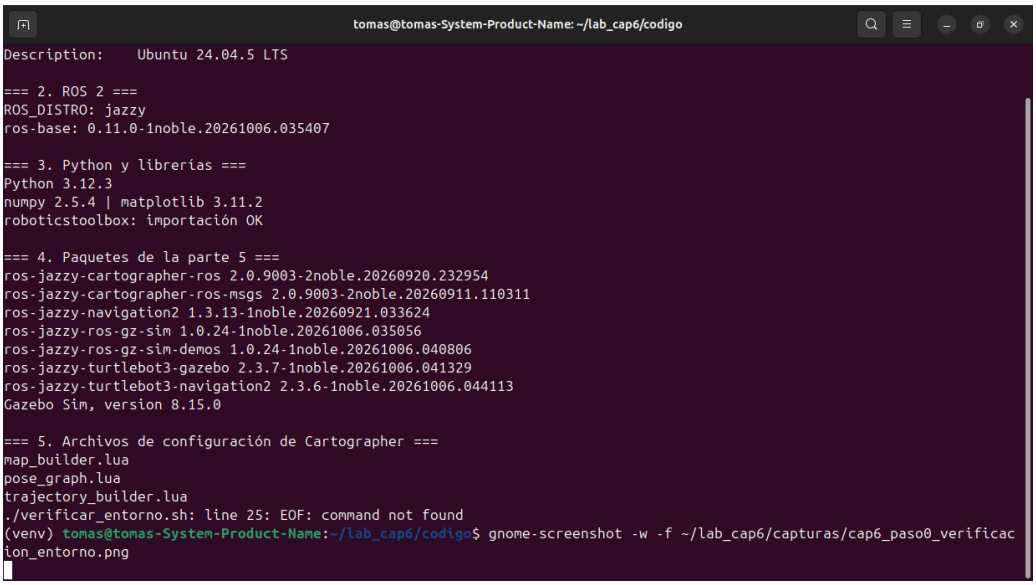


Figura 1: Verificación del entorno: sistema, versión de ROS 2 y librerías de Python.

4. Parte 1: Odometría (Dead Reckoning)

4.1. ¿Qué hicimos?

Simulamos un robot con forma de bicicleta que avanza a **1 m/s** y gira a **0.2 rad/s**. Hicimos dos robots que se mueven igual:

- Uno **con ruido**, como ocurre en la vida real: las ruedas nunca son perfectas.
- Otro **ideal**, sin ruido.

Ambos recorrieron 200 pasos y comparamos sus trayectorias. El ruido se definió con la matriz V, que indica cuánta incertidumbre hay en la velocidad lineal (0,2 m/s) y en la velocidad de giro (0,5°).

4.2. Comando y código

En la terminal, dentro de la carpeta del código y con el entorno virtual activo, ejecutamos:

```
cd ~/lab_cap6/codigo
source ~/lab_cap6/venv/bin/activate
python parte1_odometria.py
```

Listing 2: Comando de la parte 1

```
from roboticstoolbox import Bicycle
import numpy as np
import matplotlib.pyplot as plt

# Covarianza del ruido: incertidumbre para estimacion de pose
V = np.diag([0.2, np.deg2rad(0.5)])**2
print(V)

# Robot con ruido
robot = Bicycle(covar=V)

# Robot ideal (sin ruido)
robot_ideal = Bicycle(covar=np.zeros((2, 2)))

# Variables para almacenar trayectorias
x_real, y_real = [], []
x_ideal, y_ideal = [], []

qp_real = [0, 0, 0]
qp_ideal = [0, 0, 0]

# Simulacion
for _ in range(200):
    odo_real = robot.step((1.0, 0.2))
    qp_real = robot.f(qp_real, odo_real)

    odo_ideal = robot_ideal.step((1.0, 0.2))
    qp_ideal = robot_ideal.f(qp_ideal, odo_ideal)

    x_real.append(qp_real[0])
    y_real.append(qp_real[1])
    x_ideal.append(qp_ideal[0])
    y_ideal.append(qp_ideal[1])

# Graficar trayectorias
plt.figure(figsize=(8, 6))
plt.plot(x_ideal, y_ideal,
```

```
        label='Trayectoria estimada con odometria sin ruido', linewidth=1)
plt.plot(x_real, y_real,
        label='Trayectoria estimada con odometria con ruido', linewidth=1)
plt.xlabel('X [m]')
plt.ylabel('Y [m]')
plt.title('Comparacion de trayectorias')
plt.grid(True)
plt.axis('equal')
plt.legend()
plt.savefig('/home/tomas/lab_cap6/capturas/cap6_parte1_odometria.png', dpi=150)
plt.show()
```

Listing 3: Script `parte1_odometria.py`

Nota: la guía usaba `robot.f` también para el robot ideal. Lo cambiamos a `robot_ideal.f`, que es el objeto correcto. El resultado no cambia, porque ambos robots usan el mismo modelo de movimiento.

4.3. Resultado

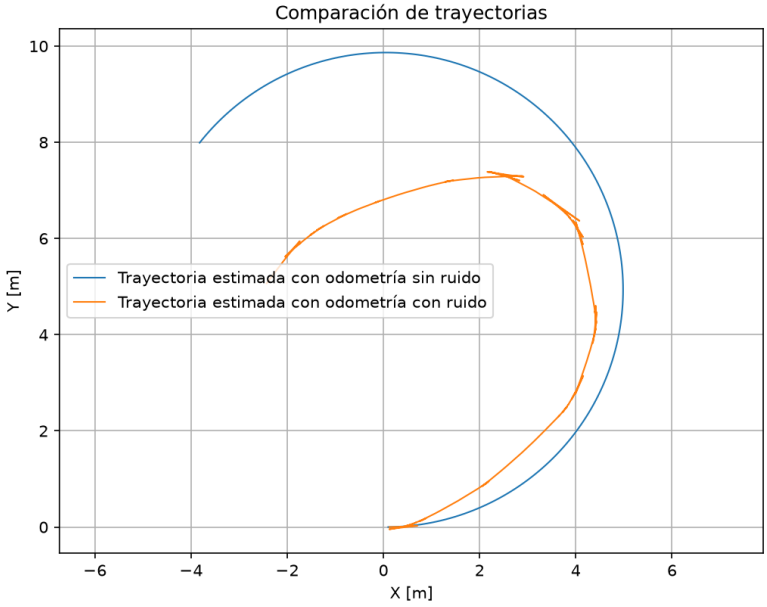


Figura 2: Trayectoria ideal (azul) y trayectoria con ruido (naranja) de la odometría.

La matriz V impresa por la terminal fue `diag(0.04, 7.6e-05)`, que corresponde a $0{,}2^2$ y $(0{,}5°)^2$ en radianes. Al inicio las dos trayectorias coinciden. Con el tiempo, la trayectoria con ruido se separa y termina lejos de la ideal. Como la velocidad es constante, la trayectoria ideal forma un círculo de radio $v/\omega = 1/0{,}2 = 5$ m.

4.4. Preguntas de análisis

1. **¿Por qué la trayectoria presenta error?**
   La odometría solo sabe cuánto creyó moverse. Cada pequeño error de las ruedas se suma al siguiente paso, y el robot no tiene forma de corregirlo. Por eso el error crece con el tiempo.

2. **¿Qué representa la matriz de covarianza V?**
   Representa la incertidumbre del movimiento. La diagonal indica cuánto puede fallar cada

comando: 0,2 m/s en la velocidad lineal y 0,5° en la velocidad de giro. Los valores son el cuadrado de esas desviaciones.

3. **¿Qué ocurre si aumenta el ruido?**
   El error crece más rápido y la trayectoria se aleja más de la ideal, aunque el robot siga el mismo camino en teoría.

5. Parte 2: Estimación con EKF

5.1. ¿Qué hicimos?

Usamos el filtro de Kalman extendido para estimar la posición de un robot que sigue una trayectoria aleatoria durante **20 segundos**. El filtro solo usa la odometría, sin sensores externos. Esta vez el ruido de la odometría es menor (0,02 en la velocidad lineal) y la incertidumbre inicial es P_0.

5.2. Comando y código

```
cd ~/lab_cap6/codigo
source ~/lab_cap6/venv/bin/activate
python parte2_ekf.py
```

Listing 4: Comando de la parte 2

```
from roboticstoolbox import Bicycle, EKF, RandomPath
import numpy as np
import matplotlib.pyplot as plt

# Covarianza odometria
V = np.diag([0.02, np.deg2rad(0.5)])**2

# Robot
robot = Bicycle(covar=V, animation=None)

# Trayectoria aleatoria
robot.control = RandomPath(workspace=10)

# Covarianza inicial
P0 = np.diag([0.05, 0.05, np.deg2rad(0.5)])**2

# EKF
ekf = EKF(robot=(robot, V), P0=P0)

# Ejecutar simulacion
ekf.run(T=20)

# Trayectoria
plt.figure(figsize=(8, 6))
robot.plot_xy(color='blue')
ekf.plot_xy(color='red')
plt.title('Trayectoria real vs estimada')
plt.grid(True)
plt.savefig('/home/tomas/lab_cap6/capturas/cap6_parte2_ekf_trayectoria.png', dpi
    =150)

# Incertidumbre
plt.figure(figsize=(8, 6))
plt.plot(ekf.get_t(), ekf.get_Pnorm(), linewidth=2)
plt.xlabel('Tiempo [s]')
plt.ylabel('Incertidumbre')
plt.title('Crecimiento de incertidumbre')
plt.grid(True)
plt.savefig('/home/tomas/lab_cap6/capturas/cap6_parte2_ekf_incertidumbre.png',
    dpi=150)

plt.show()
```

Listing 5: Script `parte2_ekf.py`

5.3. Resultado

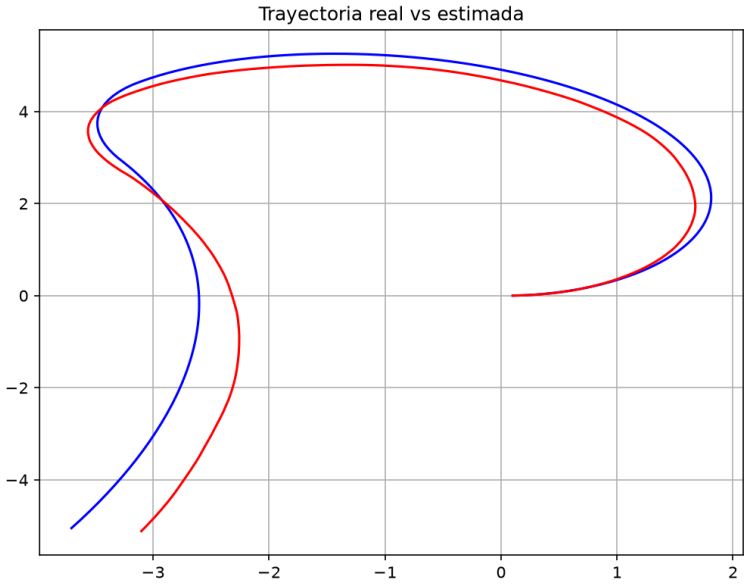


Figura 3: Trayectoria real (azul) frente a la estimada por el EKF (rojo).

La trayectoria estimada (roja) se separa poco a poco de la real (azul).

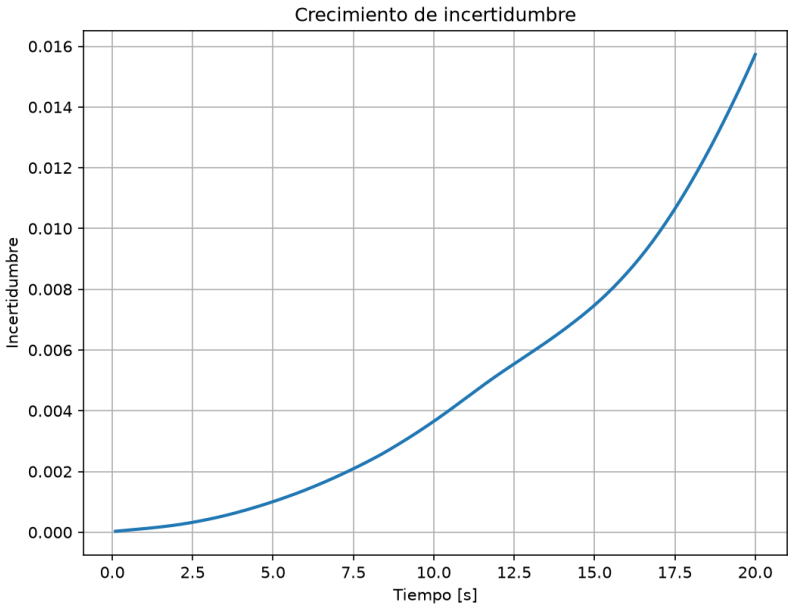


Figura 4: Incertidumbre del EKF en el tiempo, sin correcciones externas.

La incertidumbre sube de forma continua desde casi cero hasta unos 0,016 al final de los 20 segundos. Nunca baja, porque el filtro no recibe ninguna medición que lo corrija.

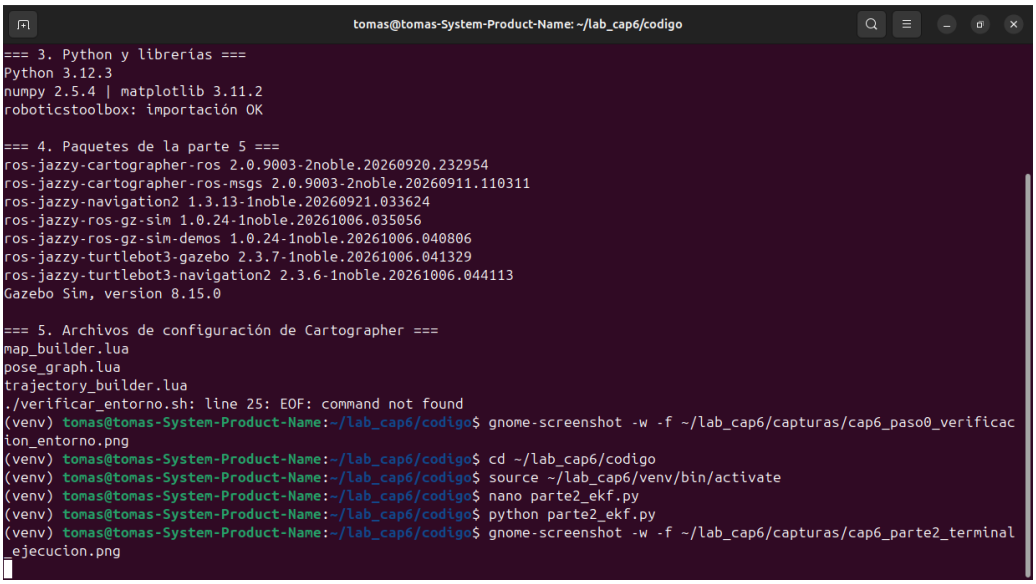


Figura 5: Ejecución de `parte2_ekf.py` en la terminal.

5.4. Preguntas de análisis

1. **¿Por qué el EKF no logra corregir el error?**
   Porque solo usa la odometría, que es la misma fuente de error. El filtro predice el movimiento y lo actualiza, pero sin una referencia externa no tiene cómo saber qué tan lejos está de la posición real.

2. **¿Qué representa la curva de incertidumbre?**
   Es qué tan seguro está el filtro de su estimación. Un valor más alto significa menos confianza. Como la curva sube sin parar, el robot cada vez sabe menos de dónde está.

3. **¿Qué ocurre si aumenta el valor de V?**
   La incertidumbre crece más rápido, porque el filtro asume que cada movimiento es menos preciso.

6. Parte 3: EKF con landmarks

6.1. ¿Qué hicimos?

Agregamos un mapa con **20 landmarks** en un espacio de 10 m. El robot tiene un sensor tipo LiDAR que ve hasta **4 m** de distancia y en un ángulo de $-90°$ a $90°$ frente a él. Cuando el sensor ve un landmark, el filtro corrige su estimación. La simulación duró **40 segundos**.

6.2. Comando y código

```
cd ~/lab_cap6/codigo
source ~/lab_cap6/venv/bin/activate
python parte3_landmarks.py
```

Listing 6: Comando de la parte 3

```
from roboticstoolbox import *
import numpy as np
import matplotlib.pyplot as plt
from math import pi

# Covarianza odometria
V = np.diag([0.02, np.deg2rad(0.5)])**2

# Robot
robot = Bicycle(covar=V, animation=None)
robot.control = RandomPath(workspace=10)

# Mapa de landmarks
mapa = LandmarkMap(20, workspace=10)

# Covarianza sensor de rango
W = np.diag([0.1, np.deg2rad(1)])**2

# Sensor de rango, tipo Lidar
sensor = RangeBearingSensor(
    robot=robot,
    map=mapa,
    covar=W,
    angle=[-pi/2, pi/2],
    range=4
)

# Covarianza inicial
P0 = np.diag([0.05, 0.05, np.deg2rad(0.5)])**2

# EKF con mapa conocido
ekf = EKF(
    robot=(robot, V),
    P0=P0,
    map=mapa,
    sensor=(sensor, W)
)

# Simulacion
ekf.run(T=40)

# Trayectorias
plt.figure(figsize=(8, 6))
mapa.plot()
robot.plot_xy(color='blue')
ekf.plot_xy(color='red')
```

```
ekf.plot_ellipse()
plt.title('EKF con landmarks')
plt.grid(True)
plt.savefig('/home/tomas/lab_cap6/capturas/cap6_parte3_landmarks_trayectoria.png', dpi=150)

# Incertidumbre
plt.figure(figsize=(8, 6))
plt.plot(ekf.get_t(), ekf.get_Pnorm(), linewidth=2)
plt.xlabel('Tiempo [s]')
plt.ylabel('Incertidumbre')
plt.title('Incertidumbre con correcciones')
plt.grid(True)
plt.savefig('/home/tomas/lab_cap6/capturas/cap6_parte3_landmarks_incertidumbre.png', dpi=150)

plt.show()
```

Listing 7: Script `parte3_landmarks.py`

6.3. Resultado

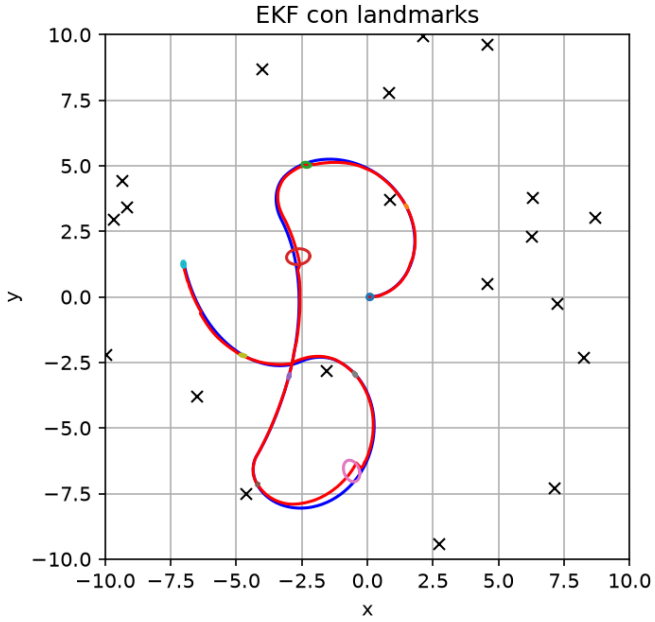


Figura 6: Mapa con landmarks (x), trayectoria real (azul), estimada (rojo) y elipses de confianza.

Ahora la trayectoria estimada (roja) sigue muy de cerca a la real (azul) durante todo el recorrido. Las elipses son pequeñas, lo que indica que el filtro está seguro de su posición.

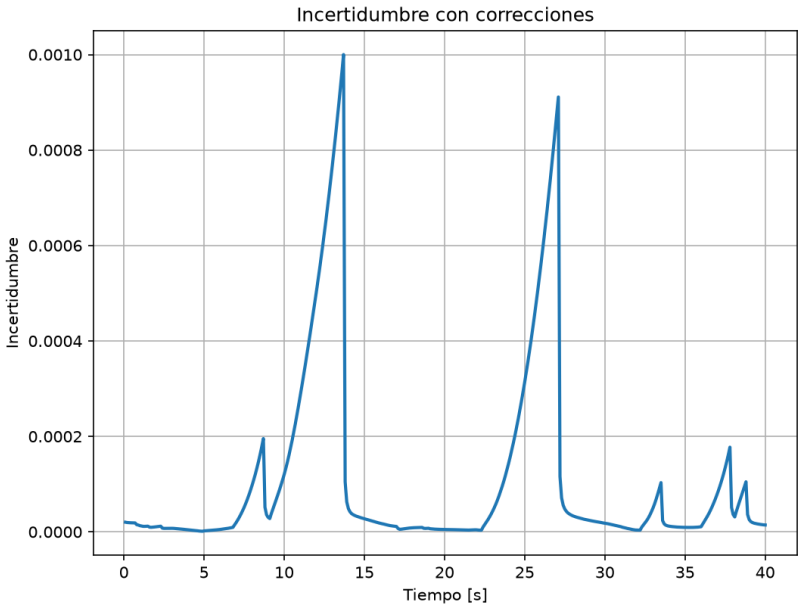


Figura 7: Incertidumbre del EKF con correcciones por landmarks.

La incertidumbre no sube sin control: sube entre observaciones y cae de golpe cada vez que el sensor ve un landmark, alrededor de los 8,7, 13,7, 27, 33,5, 37,5 y 38,8 segundos. Su valor máximo es cerca de 0,001, mucho menor que los 0,016 de la Parte 2.

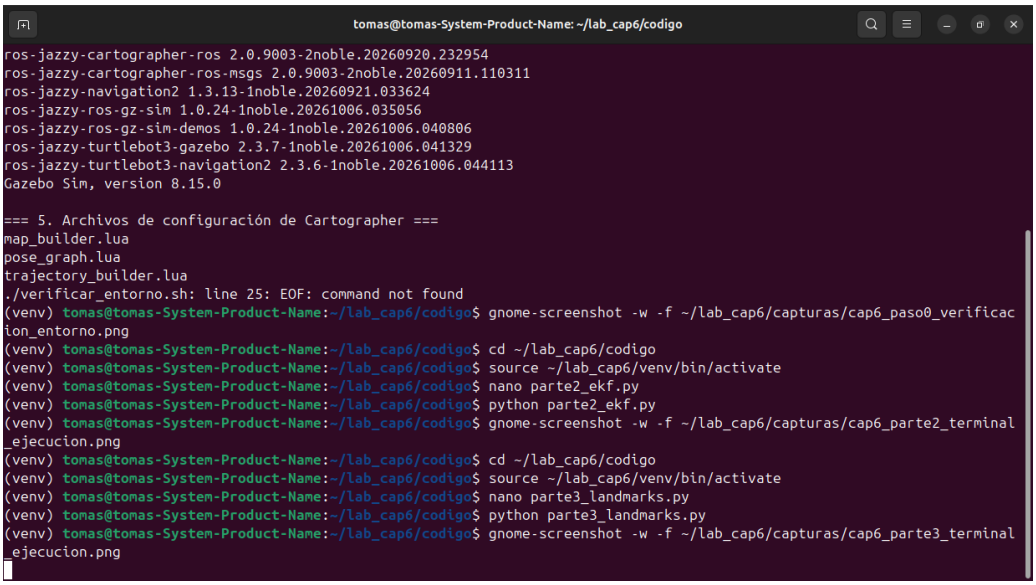


Figura 8: Ejecución de `parte3_landmarks.py` en la terminal.

6.4. Preguntas de análisis

1. **¿Qué es la innovación?**
   Es la diferencia entre lo que el sensor midió realmente y lo que el filtro esperaba medir. Si la diferencia es grande, el filtro corrige más su estimación.

2. **¿Por qué disminuye la incertidumbre?**
   Cada landmark observado aporta información nueva e independiente de la odometría. Con

esa referencia, el filtro corrige la posición y se vuelve más seguro.

3. **¿Qué ocurre cuando no se observan landmarks?**
   La incertidumbre vuelve a crecer, igual que en la Parte 2, hasta que el robot ve otro landmark. Por eso la curva tiene subidas y bajadas.

7. Parte 4: SLAM con EKF

7.1. ¿Qué hicimos?

Repetimos la simulación de la Parte 3, pero ahora el filtro **no conoce el mapa**. El robot debe construir el mapa de landmarks y, al mismo tiempo, estimar su propia posición. La simulación duró 40 segundos.

7.2. Comando y código

```
cd ~/lab_cap6/codigo
source ~/lab_cap6/venv/bin/activate
python parte4_slam.py
```

Listing 8: Comando de la parte 4

```
from roboticstoolbox import *
import numpy as np
import matplotlib.pyplot as plt
from math import pi

# Covarianza de odometria
V = np.diag([0.02, np.deg2rad(0.5)])**2

# Robot
robot = Bicycle(covar=V, animation=None)
robot.control = RandomPath(workspace=10)

# Mapa real (solo para graficar; el EKF SLAM no lo usa)
mapa = LandmarkMap(20, workspace=10)

# Sensor
W = np.diag([0.1, np.deg2rad(1)])**2
sensor = RangeBearingSensor(
    robot=robot,
    map=mapa,
    covar=W,
    angle=[-pi/2, pi/2],
    range=4
)

# Covarianza inicial
P0 = np.diag([0.05, 0.05, np.deg2rad(0.5)])**2

# EKF SLAM: no conoce el mapa
slam = EKF(
    robot=(robot, V),
    P0=P0,
    sensor=(sensor, W),
    map=None
)

# Ejecutar simulacion
slam.run(T=40)

# Trayectorias y mapa
plt.figure(figsize=(10, 8))
mapa.plot()
robot.plot_xy(color='blue', label='Trayectoria real')
slam.plot_xy(color='red', label='Trayectoria estimada')
slam.plot_ellipse()
plt.title('SLAM usando EKF')
```

```
plt.grid(True)
plt.axis('equal')
plt.legend()
plt.savefig('/home/tomas/lab_cap6/capturas/cap6_parte4_slam_trayectoria.png',
    dpi=150)

# Incertidumbre
plt.figure(figsize=(8, 6))
plt.plot(slam.get_t(), slam.get_Pnorm(), linewidth=2)
plt.xlabel('Tiempo [s]')
plt.ylabel('Incertidumbre')
plt.title('Evolucion de incertidumbre en SLAM')
plt.grid(True)
plt.savefig('/home/tomas/lab_cap6/capturas/cap6_parte4_slam_incertidumbre.png',
    dpi=150)

plt.show()
```

Listing 9: Script `parte4_slam.py`

7.3. Resultado

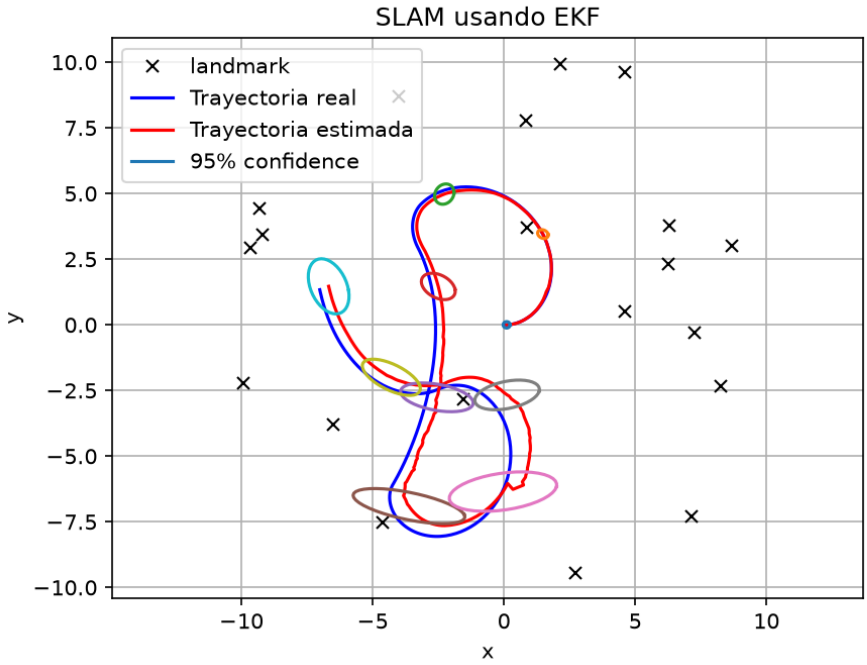


Figura 9: SLAM: landmarks estimados con elipses, trayectoria real (azul) y estimada (rojo).

Al inicio, la trayectoria estimada se separa un poco de la real, porque el robot todavía no conoce ningún landmark. A medida que los detecta, el filtro construye el mapa y la estimación se acerca a la trayectoria real. Las elipses marcan la incertidumbre de cada landmark estimado: cuando el robot pasa cerca de uno, su elipse se vuelve pequeña.

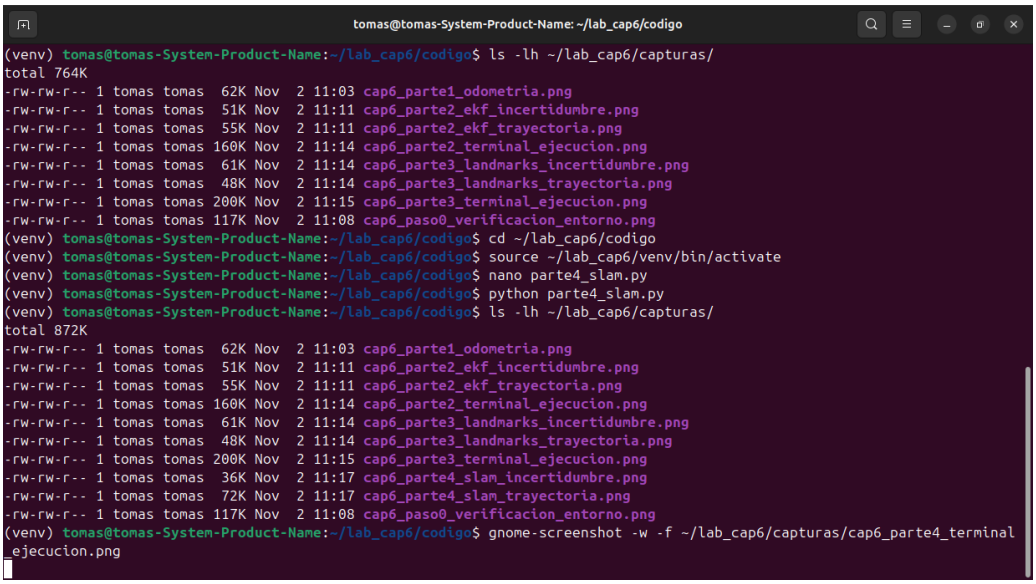


Figura 10: Ejecución de `parte4_slam.py` en la terminal.

7.4. Preguntas de análisis

1. **¿Por qué SLAM es más complejo que localización?**
   Porque el robot tiene que estimar dos cosas desconocidas a la vez: su posición y el mapa. Además, ambas están relacionadas: un error en la posición cambia el mapa, y un error en el mapa cambia la posición estimada.

2. **¿Qué ocurre si aumenta el ruido del sensor?**
   Las marcas del mapa quedan menos precisas, las elipses se vuelven más grandes y la trayectoria estimada se separa más de la real.

3. **¿Por qué la incertidumbre no desaparece completamente?**
   Porque siempre hay ruido en la odometría y en el sensor. Aunque el filtro mejora con cada observación, nunca puede estar completamente seguro. En la gráfica el valor llega a ser muy pequeño, pero no exactamente cero.

4. **¿Qué ventajas ofrece SLAM frente a usar un mapa conocido?**
   No necesita un mapa previo. Sirve para explorar lugares desconocidos, como un edificio nuevo o una zona de rescate, y construye el mapa mientras se mueve.

5. **¿Qué aplicaciones industriales podrían usar SLAM?**
   Robots de almacenes que transportan mercancía, robots de limpieza, vehículos de inspección en minas o tuberías, robots agrícolas y robots de reparto dentro de edificios.

6. **¿Cómo es la comparación de los resultados, conociendo y sin conocer el mapa?**
   Con el mapa conocido (Parte 3), el robot se ubica desde el inicio y la estimación sigue muy de cerca a la real. Sin el mapa (Parte 4), el robot empieza con un error y lo reduce a medida que construye el mapa. El resultado final es parecido, pero el SLAM necesita más trabajo: estimar el mapa y la posición al mismo tiempo.

8. Parte 5: SLAM con Cartographer (ROS 2 Jazzy, TurtleBot3, Gazebo y RViz2)

8.1. ¿Qué hicimos?

En esta parte pasamos del SLAM en Python a un SLAM con sensores simulados. El robot TurtleBot3 (modelo *burger*) se mueve dentro de la casa de Gazebo (`turtlebot3_house`), su LiDAR mide las paredes y Cartographer construye un mapa que vemos crecer en RViz2.

Seguimos los pasos de la guía (8.1 a 8.26), con tres diferencias:

1. **Empezamos desde cero.** Borramos el workspace de intentos anteriores y creamos una carpeta nueva para el laboratorio.
2. **Usamos teleoperación.** En vez del controlador simple de la guía, movimos el robot con el teclado.
3. **Cambiamos la configuración de Cartographer.** La configuración de la guía hacía que el mapa saliera con paredes duplicadas. Más adelante se explica cuál era el problema y cómo lo arreglamos.

El video de la prueba y el código están al final de esta parte.

8.2. Carpetas del laboratorio y capturas

Para tener todo ordenado (y poder subirlo después a GitHub) creamos esta estructura:

```
LAB=~/lab_cap6_slam
mkdir -p $LAB/{capturas,codigo,videos,mapas}

# Captura de pantalla completa, 5 s despues del comando
captura() { sleep 5; gnome-screenshot -f "$LAB/capturas/$1.png"; echo "Guardada:
    $1.png"; }

# Captura solo de la ventana activa (la terminal)
capturaterm() { sleep 5; gnome-screenshot -w -f "$LAB/capturas/$1.png"; echo "
    Guardada: $1.png"; }
```

Listing 10: Carpetas del laboratorio y funciones de captura

Las carpetas son `capturas/` (imágenes del informe), `codigo/` (archivos usados), `videos/` y `mapas/` (el mapa guardado). Cada captura se llama `cap6_s8_NN_descripcion.png`, para que sea fácil ubicarla en este informe.

8.3. Pasos 8.1 a 8.6: verificar el entorno

Primero comprobamos que todo estuviera instalado: ROS 2 Jazzy, Gazebo Harmonic, TurtleBot3, Cartographer y Nav2. Estos comandos nos sirvieron también como evidencia de que el equipo estaba listo.

```
ros2 --version
echo $ROS_DISTRO
gz sim --version
ros2 pkg list | grep -E "^(turtlebot3_gazebo|turtlebot3_teleop|cartographer_ros|
    nav2_map_server)$"
```

Listing 11: Verificación del entorno

La respuesta esperada de `echo $ROS_DISTRO` es `jazzy`. Si faltaba algún paquete, se instalaba con los comandos de la guía:

```
sudo apt update
sudo apt install -y ros-jazzy-turtlebot3* ros-jazzy-ros-gz ros-jazzy-ros-gz-
    bridge
sudo apt install -y ros-jazzy-cartographer ros-jazzy-cartographer-ros
sudo apt install -y ros-jazzy-navigation2 ros-jazzy-nav2-bringup ros-jazzy-nav2-
    map-server
```

Listing 12: Instalación de dependencias (solo si falta algo)

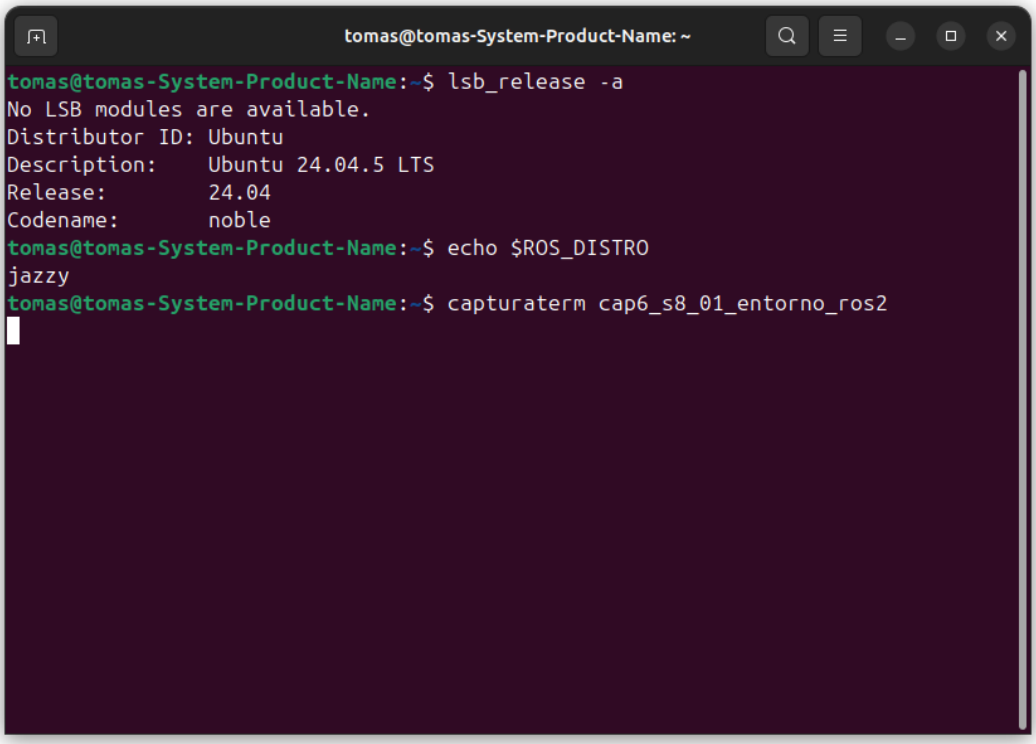


Figura 11: Versión de ROS 2 y distribución activa (`jazzy`).

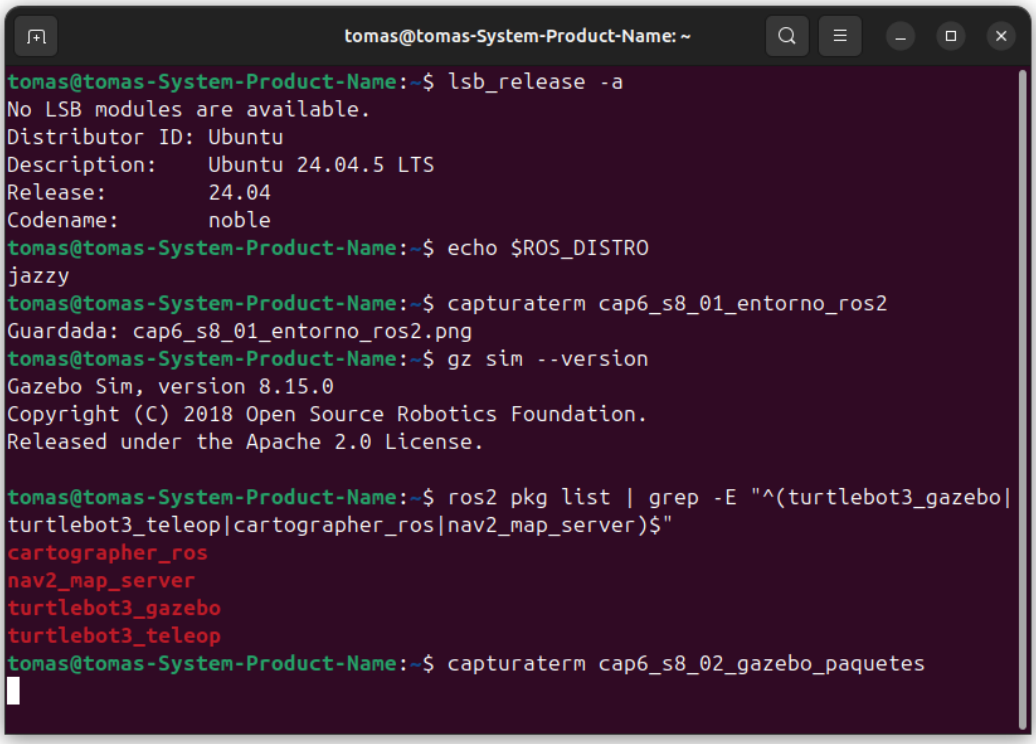


Figura 12: Versión de Gazebo Harmonic.

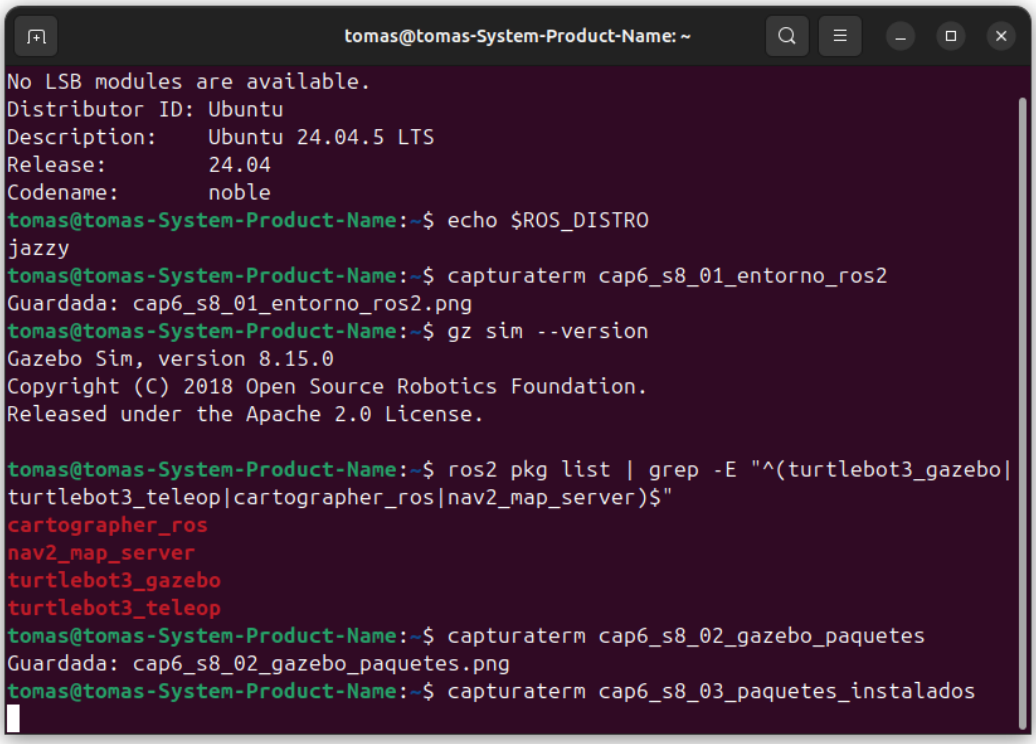


Figura 13: Paquetes de TurtleBot3, Cartographer y Nav2 encontrados.

8.4. Pasos 8.7 a 8.9: workspace y paquete

Para empezar desde cero, borramos el workspace anterior y creamos uno nuevo con el paquete `localization_lab`. Solo se borra la carpeta de trabajo; los paquetes de ROS instalados en el sistema no se tocan.

```
rm -rf ~/ros2_ws
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
ros2 pkg create localization_lab --build-type ament_python \
  --dependencies rclpy geometry_msgs sensor_msgs nav_msgs cartographer_ros
cd localization_lab
mkdir -p launch config rviz
```

Listing 13: Workspace y paquete

Cartographer necesita tres archivos de configuración base, que copiamos del sistema a nuestro paquete:

```
cp /opt/ros/jazzy/share/cartographer/configuration_files/map_builder.lua ~/
    ros2_ws/src/localization_lab/config/
cp /opt/ros/jazzy/share/cartographer/configuration_files/trajectory_builder.lua
    ~/ros2_ws/src/localization_lab/config/
cp /opt/ros/jazzy/share/cartographer/configuration_files/pose_graph.lua ~/
    ros2_ws/src/localization_lab/config/
```

Listing 14: Copia de los archivos base de Cartographer

La estructura que usamos es la de la guía:

```
localization_lab/
|-- package.xml
|-- setup.py
|-- setup.cfg
|-- resource/localization_lab
|-- localization_lab/
|   |-- __init__.py
|-- launch/
|   |-- cartographer.launch.py
|-- config/
|   |-- cartographer.lua   (y map_builder, trajectory_builder, pose_graph)
|-- rviz/
```

Listing 15: Estructura del paquete

8.5. Paso 8.10: el controlador (teleoperación)

La guía propone un nodo `simple_controller` que publica siempre la misma velocidad (0.2 m/s y 0.1 rad/s) sin mirar el entorno. Dentro de la casa eso hace que el robot choque con las paredes. Por eso lo reemplazamos por el nodo de teleoperación de TurtleBot3, que mueve el robot con el teclado:

```
ros2 run turtlebot3_teleop teleop_keyboard
```

Listing 16: Teleoperación con el teclado

Las teclas son `w` (avanzar), `x` (retroceder), `a` y `d` (girar) y `s` (detener). Como ya no hay un nodo propio, el `setup.py` no necesita ejecutables.

Problema con la teleoperación. Al principio presionábamos las teclas y el robot no se movía ni aparecía nada en la terminal. La causa era que la terminal del teleop no tenía el foco: hay que hacer clic dentro de esa ventana antes de presionar las teclas. Con eso funcionó.

8.6. Pasos 8.11 a 8.14: archivos del paquete

setup.py. Registra las carpetas `launch`, `config` y `rviz` para que se instalen con el paquete. La parte importante es esta:

```
data_files=[
    ('share/ament_index/resource_index/packages', ['resource/' + package_name]),
    ('share/' + package_name, ['package.xml']),
    (os.path.join('share', package_name, 'launch'), glob('launch/*.launch.py')),
    (os.path.join('share', package_name, 'config'), glob('config/*.lua')),
    (os.path.join('share', package_name, 'rviz'), glob('rviz/*.rviz')),
],
```

Listing 17: `setup.py`: archivos de datos

package.xml. Declara las dependencias del paquete:

```
<depend>rclpy</depend>
<depend>geometry_msgs</depend>
<depend>sensor_msgs</depend>
<depend>nav_msgs</depend>
<depend>cartographer_ros</depend>
```

Listing 18: `package.xml`: dependencias

cartographer.lua. Es el archivo que le dice a Cartographer qué sensores usar y cómo se llaman los marcos de referencia. Esta es la **versión final**, con los cambios que explicamos más adelante:

```
include "map_builder.lua"
include "trajectory_builder.lua"

options = {
  map_builder = MAP_BUILDER,
  trajectory_builder = TRAJECTORY_BUILDER,
  map_frame = "map",
  tracking_frame = "base_footprint",
  published_frame = "odom",
  odom_frame = "odom",
  provide_odom_frame = false,
  publish_frame_projected_to_2d = true,
  use_odometry = true,
  use_nav_sat = false,
  use_landmarks = false,
  num_laser_scans = 1,
  num_multi_echo_laser_scans = 0,
  num_subdivisions_per_laser_scan = 1,
  num_point_clouds = 0,
  lookup_transform_timeout_sec = 0.2,
  submap_publish_period_sec = 0.3,
  pose_publish_period_sec = 0.005,
  trajectory_publish_period_sec = 0.03,
  rangefinder_sampling_ratio = 1.0,
  odometry_sampling_ratio = 1.0,
  fixed_frame_pose_sampling_ratio = 1.0,
  imu_sampling_ratio = 1.0,
  landmarks_sampling_ratio = 1.0,
}

MAP_BUILDER.use_trajectory_builder_2d = true
TRAJECTORY_BUILDER_2D.use_imu_data = false
TRAJECTORY_BUILDER_2D.min_range = 0.12
TRAJECTORY_BUILDER_2D.max_range = 3.5
TRAJECTORY_BUILDER_2D.missing_data_ray_length = 3.0
TRAJECTORY_BUILDER_2D.use_online_correlative_scan_matching = true
```

```
TRAJECTORY_BUILDER_2D.motion_filter.max_angle_radians = math.rad(0.1)
POSE_GRAPH.constraint_builder.min_score = 0.65
POSE_GRAPH.constraint_builder.global_localization_min_score = 0.7
POSE_GRAPH.optimize_every_n_nodes = 35

return options
```

Listing 19: `cartographer.lua` (versión final)

Los parámetros más importantes son:

- `map_frame`, `tracking_frame`, `published_frame` y `odom_frame`: los nombres de los marcos de referencia. Tienen que coincidir con los que publica el robot.
- `use_odometry`: si Cartographer usa o no la odometría de las ruedas.
- `num_laser_scans = 1`: usamos un solo LiDAR (el tópico `/scan`).
- `min_range` y `max_range`: las lecturas fuera de este rango (0.12 a 3.5 m) se descartan.
- `optimize_every_n_nodes`: cada cuántos nodos se corrige el mapa completo (cierre de bucle).

cartographer.launch.py. Lanza los dos nodos de Cartographer con el tiempo de simulación de Gazebo (`use_sim_time`). El primero construye el mapa y el segundo lo convierte en una imagen (mapa de ocupación) con resolución de 0.05 m por celda.

```
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory


def generate_launch_description():
    config_dir = os.path.join(
        get_package_share_directory('localization_lab'), 'config')

    cartographer_node = Node(
        package='cartographer_ros',
        executable='cartographer_node',
        output='screen',
        parameters=[{'use_sim_time': True}],
        arguments=['-configuration_directory', config_dir,
                   '-configuration_basename', 'cartographer.lua'])

    occupancy_grid_node = Node(
        package='cartographer_ros',
        executable='cartographer_occupancy_grid_node',
        output='screen',
        parameters=[{'use_sim_time': True}],
        arguments=['-resolution', '0.05'])

    return LaunchDescription([cartographer_node, occupancy_grid_node])
```

Listing 20: `cartographer.launch.py`

8.7. Pasos 8.15 a 8.17: compilar y configurar el robot

Compilamos el workspace y cargamos el entorno. El modelo de TurtleBot3 se fija con una variable de entorno.

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
export TURTLEBOT3_MODEL=burger
```

Listing 21: Compilación y entorno

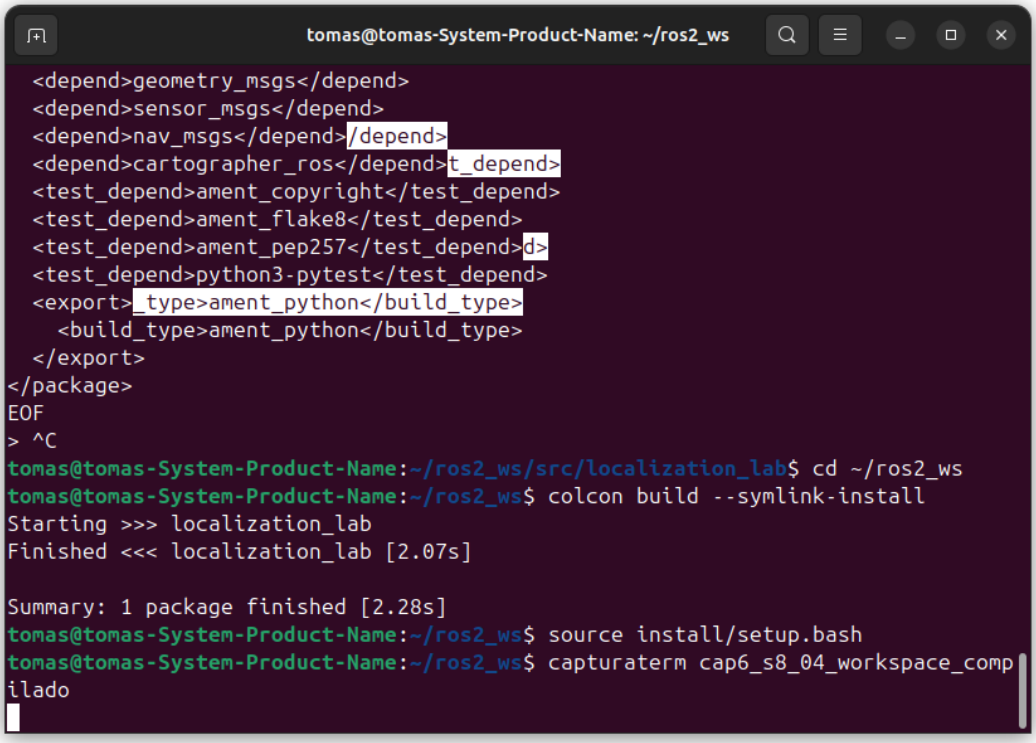


Figura 14: Compilación exitosa del paquete `localization_lab`.

8.8. Pasos 8.18 a 8.23: ejecución

Usamos cinco terminales. En cada una se cargan primero los entornos de ROS 2 y del workspace, y se define `TURTLEBOT3_MODEL`.

```
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
export TURTLEBOT3_MODEL=burger
```

Listing 22: Entorno de cada terminal

Terminal 1: Gazebo con la casa. La guía usa `empty_world`; nosotros usamos la casa, y el robot aparece en un punto dentro de ella.

```
ros2 launch turtlebot3_gazebo turtlebot3_house.launch.py x_pose:=-2.0142 y_pose
    :=0.5000
```

Listing 23: Gazebo con el mundo de la casa

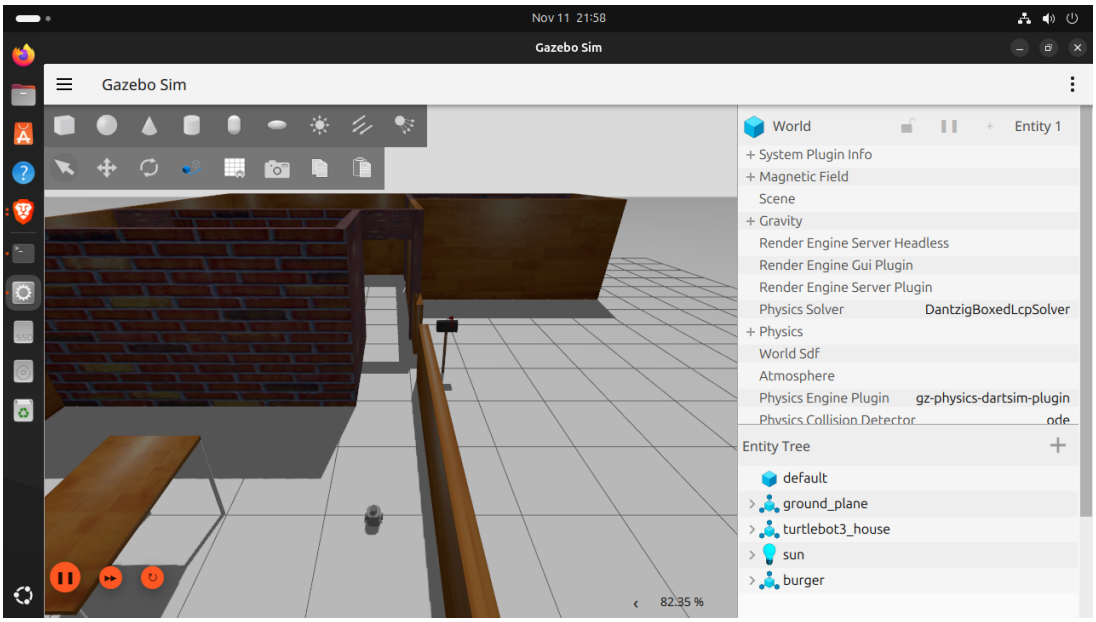


Figura 15: TurtleBot3 *burger* dentro de la casa en Gazebo Harmonic.

Terminal 2: verificar los tópicos. Antes de seguir confirmamos que el robot publica lo que Cartographer necesita: `/scan`, `/odom`, `/tf` y `/cmd_vel`.

```
ros2 topic list
ros2 topic info /cmd_vel
```

Listing 24: Verificación de tópicos

El tópico `/cmd_vel` es de tipo `geometry_msgs/msg/TwistStamped`, como indica la nota de Jazzy de la clase.

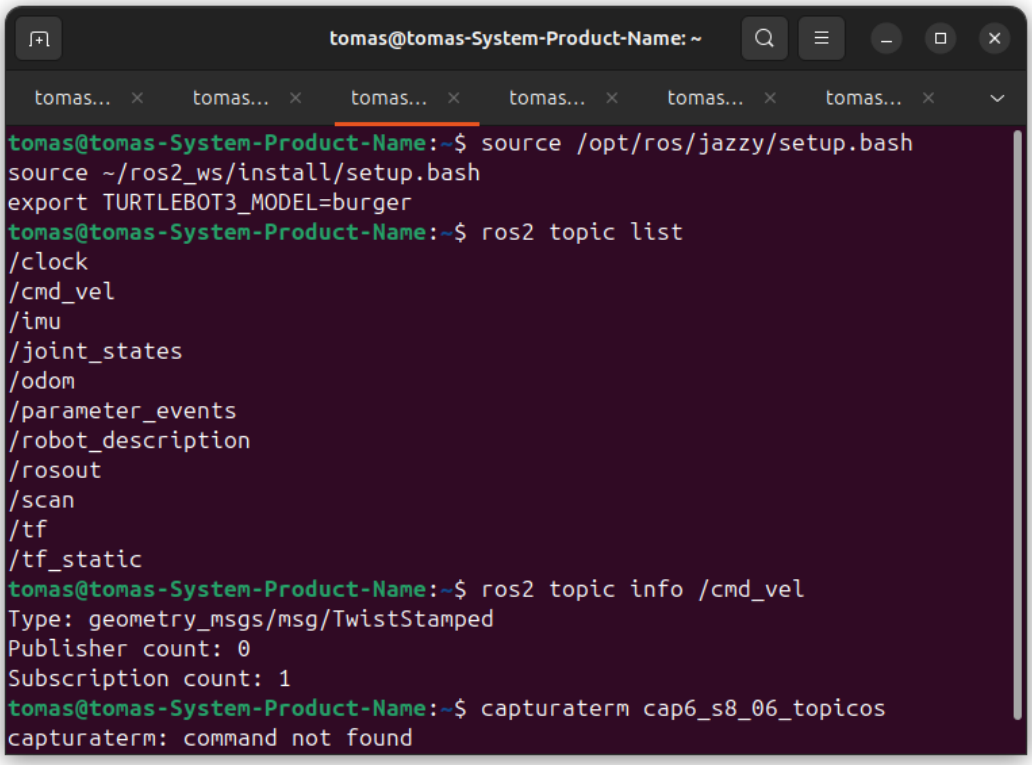


Figura 16: Lista de tópicos activos (`/scan`, `/odom`, `/tf`, `/cmd_vel`).

Falta la imagen `cap6_s8_07_cmd_vel_tipo.png`

Figura 17: Tipo de mensaje de `/cmd_vel`.

Terminal 3: teleoperación.

```
ros2 run turtlebot3_teleop teleop_keyboard --ros-args -p use_sim_time:=true
```

Listing 25: Teleoperación

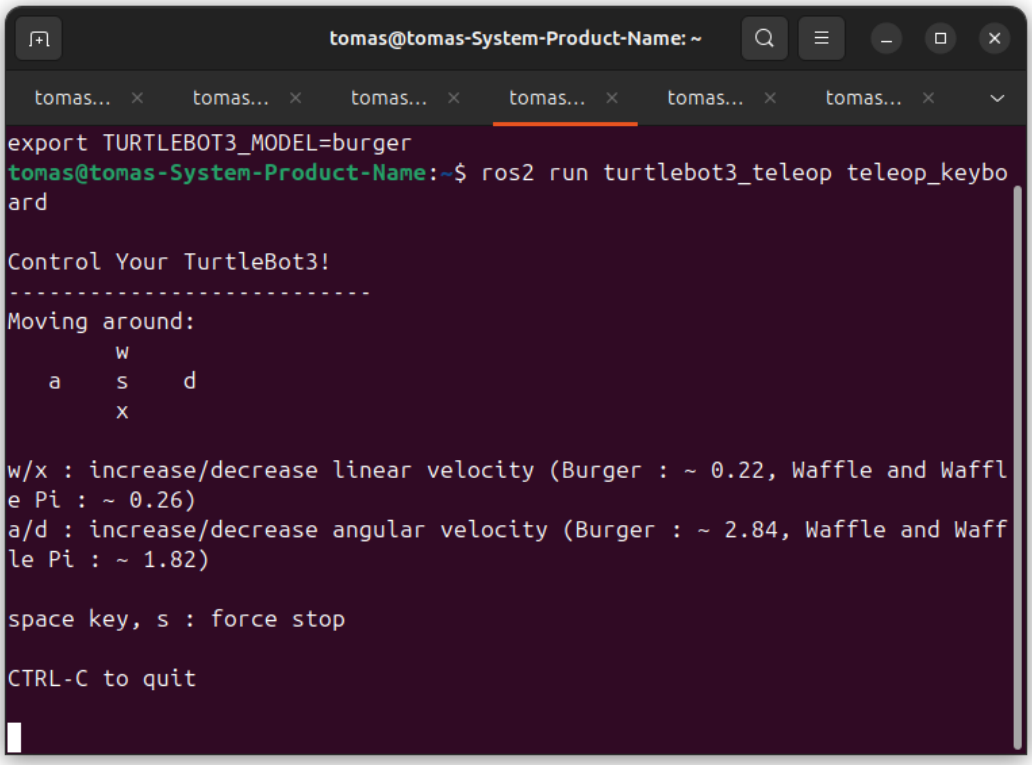


Figura 18: Nodo de teleoperación activo con el menú de teclas.

Terminal 4: Cartographer.

```
ros2 launch localization_lab cartographer.launch.py
```

Listing 26: Cartographer

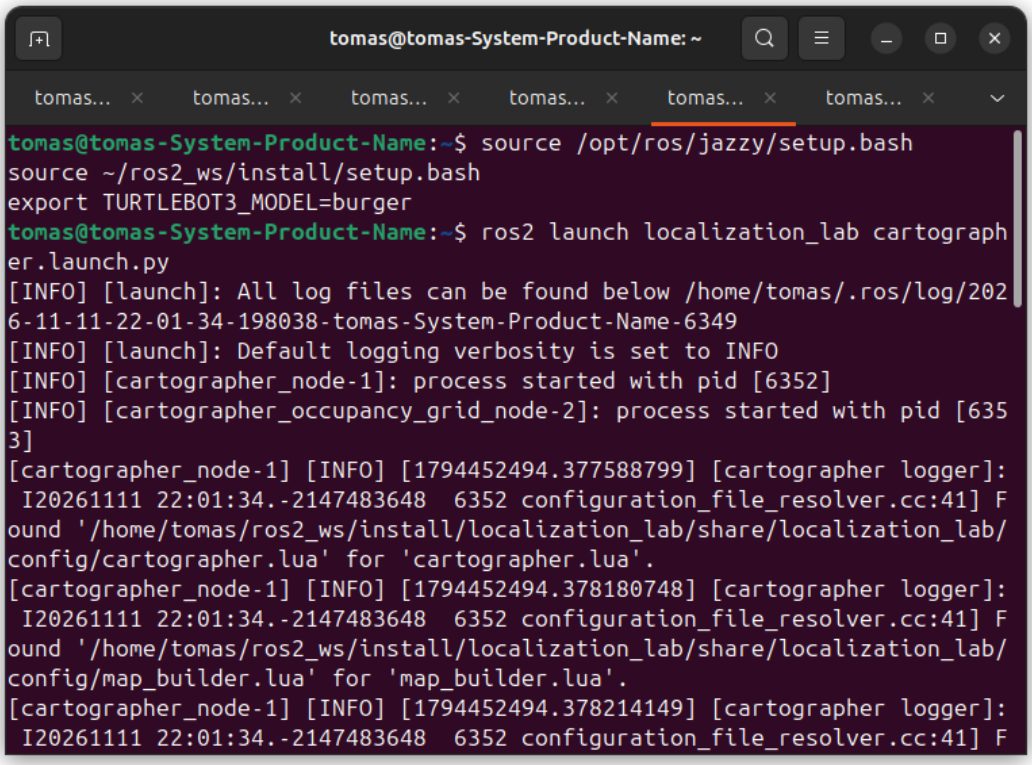


Figura 19: Inicio de los nodos de Cartographer.

Terminal 5: RViz2. Lo abrimos con el tiempo de simulación para que coincida con Gazebo.

```
ros2 run rviz2 rviz2 --ros-args -p use_sim_time:=true
```

Listing 27: RViz2

En RViz2 configuramos el *Fixed Frame* en `map` y agregamos los displays `/map`, `/scan`, `/tf` y `/submap_list`. Para que no se llenara de texto, desmarcamos *Show Names* en el display de TF.

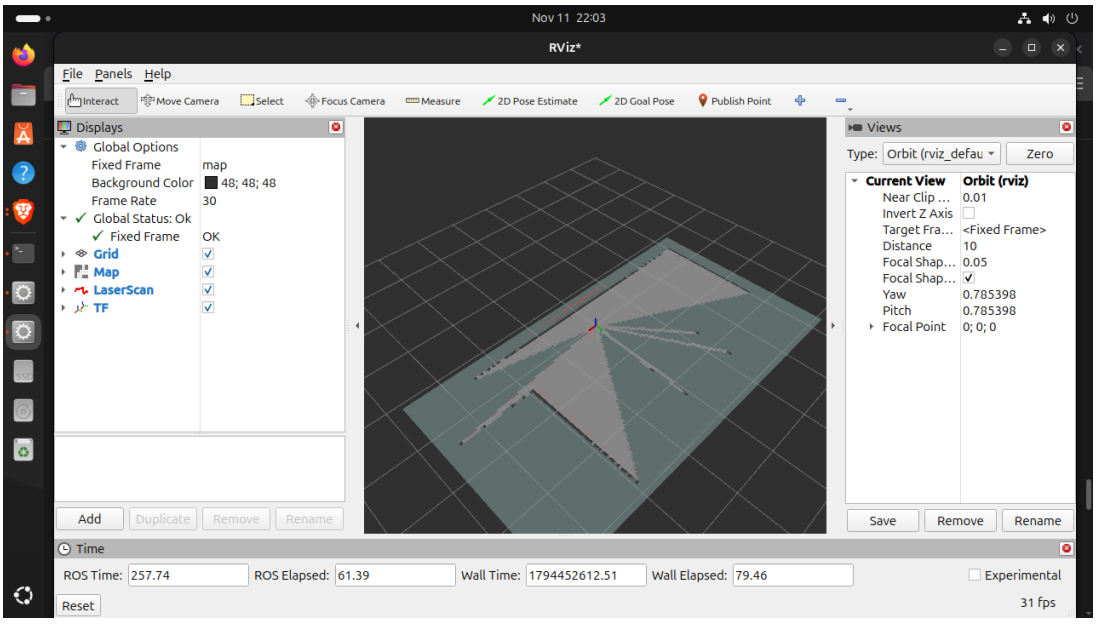


Figura 20: RViz2 configurado con *Fixed Frame* `map` y los displays del mapa y del LiDAR.

8.9. Mapeo, errores y cambios

Con todo en marcha, recorrimos la casa con el robot. Este fue el proceso real, con los errores que tuvimos:

N.°	Qué probamos	Resultado
1	Mapeo con el `cartographer.lua` de la guía y teleoperación.	El mapa salió con paredes duplicadas y objetos encima de otros.
2	Revisar que no hubiera nodos repetidos (`ps aux`, `ros2 node list` y `ros2 topic info /map -v`).	Había un solo `cartographer_node` y un solo publicador en `/map`. No era la causa.
3	Mover el robot en toques cortos de tecla, activar `use_online_correlative_scan_matching` y reiniciar todo en orden.	Mejoró poco. Después de un choque leve volvieron a duplicarse las paredes.
4	Controlador automático lento (0.05 m/s y 0.1 rad/s), con giro en el sitio.	Seguían las paredes duplicadas y los links del robot parpadeaban en RViz2.
5	Revisar los marcos TF y cambiar la configuración de Cartographer (ver tabla siguiente).	El mapa dejó de duplicarse y quedó limpio.

¿Cuál era el problema? Cada marco de referencia (*frame*) debe tener un solo "dueño" que lo publique. El marco `odom` → `base_footprint` ya lo publica Gazebo con la odometría de las ruedas. La configuración de la guía hacía que Cartographer lo publicara también (`provide_odom_frame = true` y `published_frame = base_footprint`). Con dos fuentes para el mismo marco, el robot saltaba entre dos poses distintas. Eso explica el parpadeo de los links en RViz2 y las paredes dobles en el mapa.

La solución fue que Cartographer **lea** la odometría y publique solo el marco `map` → `odom`. Estos fueron los cambios en el archivo de configuración:

Parámetro	Guía	Final
`tracking_frame`	`base_scan`	`base_footprint`
`published_frame`	`base_footprint`	`odom`
`provide_odom_frame`	`true`	`false`
`use_odometry`	`false`	`true`
`missing_data_ray_length`	`5.0`	`3.0`
`use_online_correlative_scan_matching`	no estaba	`true`
`motion_filter.max_angle_radians`	valor por defecto	`math.rad(0.1)`
`constraint_builder.min_score`	valor por defecto	`0.65`
`global_localization_min_score`	valor por defecto	`0.7`

Controlador final. Con la configuración corregida preparamos también un controlador automático (nodo `explorador`) que avanza, se detiene al ver un obstáculo y gira en el sitio hacia una dirección libre. Aun así, decidimos volver a la **teleoperación** para la prueba final, porque es lo que reemplaza al controlador de la guía y nos permite controlar que el robot no salga de la casa. Reiniciamos todo desde cero y mapeamos con toques cortos de tecla: avanzar despacio, girar de a poco y detener el robot (`s`) antes de cambiar de habitación. El código del explorador queda en el repositorio como referencia, pero no se usó en la prueba final.

Falta la imagen `cap6_s8_11_mapeo_en_progreso.png`

Figura 21: Mapa construyéndose en RViz2 mientras el robot recorre la casa en Gazebo.

8.10. Pasos 8.24 y 8.25: resultado y guardado del mapa

Al terminar el recorrido, el mapa de la casa quedó completo y sin paredes duplicadas.

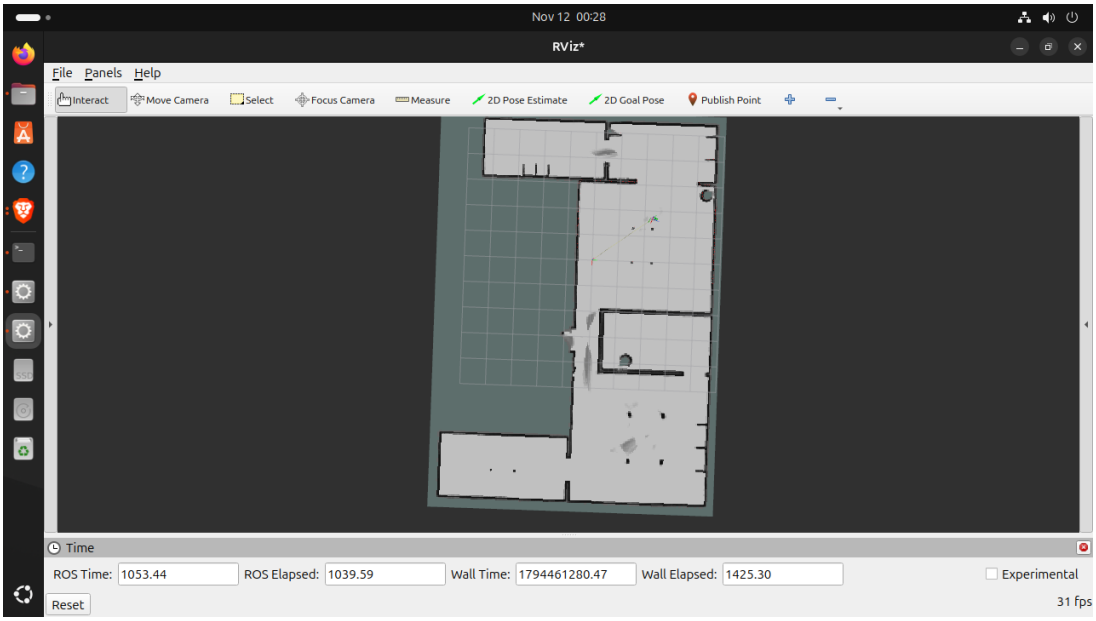


Figura 22: Mapa final de la casa construido por Cartographer, visto en RViz2.

Comparamos lo que pedía la guía como resultado esperado con lo que observamos:

Resultado esperado (guía)	**Observado**
Robot moviéndose en Gazebo	Sí, con teleoperación.
Nube de puntos del LiDAR en RViz2	Sí, con el display `/scan`.
Mapa construyéndose	Sí, con el display `/map`.
Transformaciones TF	Sí, con el display `/tf`.

Para guardar el mapa usamos el servidor de mapas de Nav2:

```
ros2 run nav2_map_server map_saver_cli -f ~/lab_cap6_slam/mapas/
    cap6_s8_mapa_casa
```

Listing 28: Guardado del mapa

El comando genera dos archivos: `cap6_s8_mapa_casa.pgm` (la imagen del mapa) y `cap6_s8_mapa_casa.yaml` (resolución, origen y umbrales). La resolución es de 0.05 m por píxel, igual que el parámetro `-resolution` del nodo. Como LaTeX no abre archivos `.pgm`, lo convertimos a PNG:

```
convert ~/lab_cap6_slam/mapas/cap6_s8_mapa_casa.pgm ~/lab_cap6_slam/capturas/
    cap6_s8_14_mapa_casa.png
```

Listing 29: Conversión del mapa a PNG

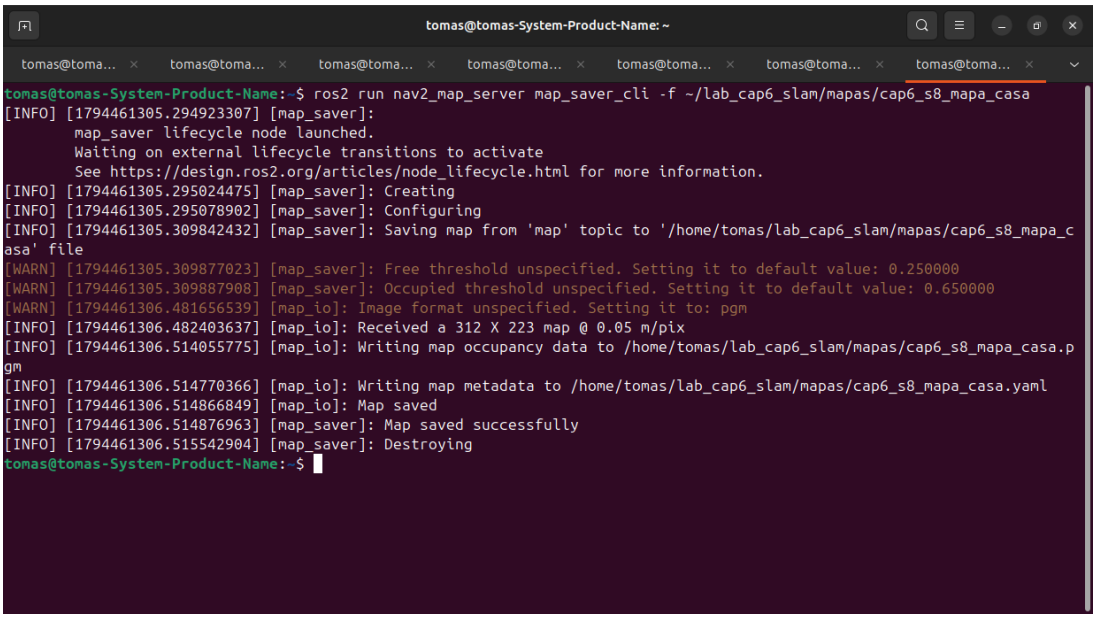


Figura 23: Guardado del mapa con `map_saver_cli`.



Figura 24: Mapa guardado (`.pgm` convertido a PNG).

8.11. Evidencia en video y código

- **Video de la práctica:** https://youtu.be/rzjI1KvMs-s
  Muestra Gazebo y RViz2 al mismo tiempo mientras el robot recorre la casa con teleoperación y el mapa se construye.
- **Repositorio con el código:** https://github.com/Rocha212/robotica-man4101136

Contiene el `cartographer.lua`, el launch file, los archivos del paquete `localization_lab`, el nodo `explorador` y el mapa guardado.

8.12. Preguntas de análisis

1. **¿Qué datos necesita Cartographer en este laboratorio y de dónde salen?**
   Necesita las lecturas del LiDAR (`/scan`), la odometría de las ruedas (`/odom`) y las transformaciones TF que dicen dónde está el LiDAR respecto al robot. Todo lo publica el TurtleBot3 simulado en Gazebo. No usamos IMU.

2. **¿Para qué sirven los marcos `map`, `odom`, `base_footprint` y `base_scan`?**
   `map` es la referencia global que construye el SLAM. `odom` es la referencia del movimiento relativo (odometría). `base_footprint` es la base del robot sobre el suelo. `base_scan` es donde está físicamente el LiDAR. Sin esa geometría, Cartographer no puede ubicar los rayos del LiDAR dentro del mapa.

3. **¿Por qué se duplicaban las paredes y cómo se solucionó?**
   Porque dos nodos (Gazebo y Cartographer) publicaban el mismo marco `odom` → `base_footprint`, y el robot saltaba entre dos poses. Se solucionó configurando Cartographer para usar la odometría (`use_odometry = true`) y publicar solo `map` → `odom` (`provide_odom_frame = false`).

4. **¿Qué diferencia hay entre SLAM local y SLAM global en Cartographer?**
   El SLAM local alinea cada nuevo escaneo con los submapas cercanos (*scan matching*) para estimar el movimiento inmediato. El SLAM global usa un grafo de poses que guarda restricciones entre submapas y, al reconocer un lugar ya visitado (cierre de bucle), corrige el error acumulado. Por eso conviene volver a pasar por zonas ya mapeadas.

5. **¿En qué se parece y en qué se diferencia de SLAM con EKF de la Parte 4?**
   Los dos estiman la posición del robot y el mapa al mismo tiempo, sin conocer el entorno. En la Parte 4 el mapa eran 20 puntos (*landmarks*) y el filtro era un EKF. Aquí el mapa es una rejilla de ocupación hecha con un LiDAR y Cartographer usa *scan matching* y optimización del grafo de poses.

6. **¿Qué ventaja tuvo la teleoperación sobre el controlador de la guía?**
   Permite controlar el recorrido, evitar choques y mover el robot despacio. El controlador de la guía avanza siempre igual y no ve las paredes. Moverse despacio ayuda, porque cada escaneo se parece más al anterior. Sin embargo, no era la causa del mapa duplicado: lo que lo arregló fue la configuración de los marcos TF.

7. **¿Qué limitaciones observamos?**
   El mapa depende de recorrer bien todo el espacio: lo que el robot no ve queda sin información. Además, un error en la configuración de los marcos TF daña el mapa aunque el algoritmo esté bien, y una simulación lenta o con mucha carga hace más difícil operar el robot con precisión.

8.13. Paso 8.26: concepto central

El sistema que armamos implementa el flujo real que usa TurtleBot3:

$$\text{LiDAR} \rightarrow \text{Cartographer} \rightarrow \text{SLAM} \rightarrow \text{Mapa} \rightarrow \text{RViz2}$$

9. Discusión comparativa

Parte	**¿Conoce el mapa?**	**¿Se corrige el error?**	**Comportamiento de la incertidumbre**
1. Odometría	No	No	Crece con el tiempo
2. EKF	No	No	Crece de forma continua hasta unos 0,016
3. EKF con landmarks	Sí	Sí, con cada observación	Sube y baja; máximo cerca de 0,001
4. SLAM	No	Sí, construye el mapa	Cae casi a cero y se mantiene plana
5. SLAM con Cartographer (ROS 2)	No	Sí, con *scan matching* y cierre de bucle	No se graficó; se evaluó la consistencia del mapa en RViz2

La conclusión general es que la odometría sola siempre acumula error. Las observaciones de landmarks son lo que permite corregirlo, ya sea con un mapa conocido (Parte 3) o construyéndolo al mismo tiempo (Parte 4).

10. Conclusiones

1. La odometría sirve para estimar la posición, pero el error se acumula con el tiempo.
2. El EKF combina predicción y medición, y calcula su propia incertidumbre. Sin medición externa, la incertidumbre crece sin límite.
3. Con landmarks conocidos, la estimación se mantiene cercana a la real y la incertidumbre se reduce cada vez que el robot observa una marca.
4. SLAM permite construir el mapa y ubicar el robot al mismo tiempo, sin conocer el entorno previamente. La escala de su incertidumbre debe verificarse antes de compararla con la de la Parte 3.
5. Con Cartographer, TurtleBot3, Gazebo y RViz2 construimos el mapa de una casa a partir de un LiDAR. El mapa solo quedó bien después de corregir la configuración de los marcos TF: cada marco debe tener un solo nodo que lo publique. El código está en https://github.com/Rocha212/robotica-man4101136.

Referencias

1. P. Corke, *Robotics, Vision and Control*, Springer. Capítulo 6: Localización y mapeo.
2. Guía de laboratorio “Simulación Capítulo 6 – Localización y Mapeo”.
3. Documentación de Cartographer ROS: https://google-cartographer-ros.readthedocs.io.
4. Robotis, TurtleBot3 e-Manual (SLAM): https://emanual.robotis.com/docs/en/platform/turtlebot3/slam/.
5. Presentación de clase 10: “CAP6 y bases de Cartographer”, Robótica 2026-2.